```
»       end
»       it "has a total price of twice the product's price" do
»         expect(cart.total_price).to eq(book_one.price * 2)
»       end
»     end
    end

  end
```

Running `bin/rails spec`, it should pass:

```
> bin/rails spec
.....

Finished in 0.11007 seconds (files took 1.72 seconds to load)
5 examples, 0 failures
```

A lot of code in this file isn't executing a test, but all the calls to `describe`, `context`, and `it` aren't for naught. Passing `SPEC_OPTS="--format=doc"` to the spec task, the test output is formatted like the documentation of the `Cart` class:

```
> bin/rails spec SPEC_OPTS="--format=doc"

Cart
  #add_product
    adding unique products
      has two line items
      has a total price of the two items' price
    adding duplicate products
      has one line item
      has a line item with a quantity of 2
      has a total price of twice the product's price

Finished in 0.14865 seconds (files took 1.76 seconds to load)
5 examples, 0 failures
```

Also note that `rspec-rails` changes the Rails generators to create empty spec files in `spec/` instead of test files in `test/`. This means that you use all the generators and scaffolding you're used to in your normal workflow without having to worry about the wrong type of test file being created.

If all of this seems strange to you, you're not alone. It *is* strange, and the reasons